Final Course Project — 2026

> **Reading guide:** sections marked **Required** are mandatory for full marks. Sections marked **Optional** describe a bonus opportunity. Sections marked **Hints** give guidance but are not prescriptive.

1. Background and Data

> [**BACKGROUND**] Read this section first. It explains the project objective, the released data, and the main constraint on the workflow.

Build a PostgreSQL data model over a software supply chain dataset derived from PyPI package metadata and OSV vulnerability advisories. Your job is to load the released JSON files into MongoDB raw collections first, ETL those raw documents into a PostgreSQL relational model, document that model clearly for both humans and Claude Sonnet 4.6, and answer a small set of analytical questions.

The grading focus is **model quality, join clarity, and documentation quality**. The cross-group LLM test matters, but it is not the main evaluation criterion.

These are the only raw data files released to student groups. *(Full field reference: see Appendix A at the end.)*

1.1 PyPI — Python Package Index

PyPI is the official third-party software repository for Python. Each document in `pypi_items.json` was fetched using the PyPI JSON API and enriched with ownership data and historical dependency declarations not available in a single API call.

Key fields to notice — you should document what you loaded into tables and what you left as raw JSON:

- `info` — package metadata: `name`, `version`, `summary`, `author_email`, `requires_dist`, `requires_python`, `license`, `classifiers`, `project_urls`
- `releases` — a dict keyed by version string; each value is a list of release-file objects containing `upload_time_iso_8601`, `filename`, `packagetype`, `size`, `python_version`, `yanked`
- `urls` — same structure as a single version inside `releases`, for the latest published version
- `ownership` — project-enrichment field; contains `organization` (nullable) and `roles` (list of `{role, user}` objects)
- `_historical_requires_dist` — project-enrichment field; a dict keyed by version string containing the `requires_dist` list declared for that historical release; this is the source of truth for dependency edges
- `vulnerabilities` — list of known vulnerability IDs linked directly by PyPI (may be empty)

1.2 OSV — Open Source Vulnerabilities

OSV is a distributed vulnerability database for open source software. Each document in `vulnerabilities.json` corresponds to one Python package and contains every advisory linked to that package.

Key fields to notice:

- `package` — the canonical package name as it appears in the advisory database
- `source` — the data source identifier (e.g. `osv-api`)
- `vulnerabilities` — array of advisory objects; each advisory contains:
  - `id` — advisory identifier (e.g. `GHSA-...`, `CVE-...`, `PYSEC-...`)
  - `summary` — one-line description
  - `details` — full advisory text (Markdown)
  - `published`, `modified` — ISO 8601 timestamps
  - `aliases` — list of cross-database identifiers
  - `severity` — list of `{type, score}` objects (CVSS)
  - `references` — list of `{type, url}` objects
  - `affected` — list of affected-package entries, each with `package`, `versions`, and `ranges`
  - `database_specific` — source-specific metadata (e.g. CWE, severity label)

1.3 Hint — Note on Excluded Packages

The released dataset contains **19,988** PyPI documents and **780** OSV documents. A small number of packages that appeared in the original upstream export were excluded before release. For example, you will not find `acryl-datahub` in either file.

> **Think about it:** what property of a package like `acryl-datahub` might cause a straightforward loader to fail when writing raw documents into MongoDB? What does this tell you about the limitations of loading large nested documents into a document store without pre-processing? Document your reasoning in the caveats section of your PDF.

---

2. Required Workflow

[**REQUIRED**] This is the core project flow. Your submission should follow this sequence.

For this project, the raw landing zone is MongoDB and the final analytical system is PostgreSQL.

Complete the project in the order below:

1. Load the two released JSON sources into MongoDB raw collections.
2. Inspect and reshape the raw MongoDB documents if needed.
3. ETL the raw MongoDB documents into a clear PostgreSQL relational model.
4. Write a prompt and data dictionary that explain the model and joins.
5. Provide one analytical question per group member (3–4 questions total) with reference answers.
6. Submit the required deliverables described in Section 3.

MongoDB is part of the required workflow, but it is **not** the final query target. Your final schema, prompt, reference queries, and answer key must target PostgreSQL.

Three starter resources are provided in the repo:

`fcp/scripts/load_json_to_mongo.py` — loads `pypi_items.json` and `vulnerabilities.json` into MongoDB, creating the collections `raw_pypi_package` and `raw_osv_package`. Run this once before writing any ETL code:

```
python fcp/scripts/load_json_to_mongo.py --drop-existing
```

`fcp/scripts/example_mongo_to_postgres_etl.py` — a minimal worked example that reads `raw_pypi_package` from MongoDB and upserts rows into one PostgreSQL table (`ssc.dim_package`). It is not a complete ETL — its purpose is to show the connection pattern, the name-normalisation helper, and the upsert idiom. Use it as a starting point for your own notebook:

```
python fcp/scripts/example_mongo_to_postgres_etl.py --truncate-target --limit 100
```

`fcp/sql/postgres_dim_package_example.sql` — an example `CREATE TABLE` statement for `ssc.dim_package`. It shows how to declare a surrogate key, a normalised-name column, and a foreign-key–ready structure. Your schema will need to extend this pattern to cover versions, vulnerabilities, and any other entities you decide to model.

You are allowed, and encouraged, to use LLM tools while doing the project. However, this must remain **human-in-the-loop** work.

- You are responsible for the final schema, notebook, prompt, questions, and answers.
- You must review generated SQL, ETL logic, and written explanations before submission.
- You should document important corrections, judgment calls, and validation checks performed by the team.

---

3. Required Deliverables and Submission Specifications

[**REQUIRED**] Everything in this section is mandatory unless a line explicitly says otherwise.

3.1 Submission Components

Submit all five components below:

1. **PostgreSQL schema file** — a standalone `.sql` file that creates all tables, constraints, and indexes
2. **ETL notebook** — a `.ipynb` file that runs end-to-end from MongoDB to PostgreSQL
3. **LLM prompt document** — a `.md` file of no more than 1,000 words describing your model for Claude Sonnet 4.6
4. **Supporting PDF** — no more than 2,000 words (excluding SQL), containing design rationale, data dictionary, ETL mapping, join rules, analytical questions with reference answers, and a contribution summary
5. **Peer-assessment file** — a short document with each member's contribution and peer evaluation

3.2 LLM Prompt Document

Your prompt must:

- be less than 1,000 words (excluding any code examples),
- state the grain of each table so aggregations are done correctly,
- identify the main join keys,
- include caveats that affect counts or grouping (e.g. packages with no versions, advisories with no affected ranges),
- optionally include one or two short join examples.

3.3 Supporting PDF

Your PDF must include:

- design rationale
- entity grain for each table
- all primary keys, foreign keys, and logical join keys
- data dictionary (mapping raw JSON fields to PostgreSQL columns) *(example mapping: see Appendix A at the end)*
- the ETL documentation from MongoDB to PostgreSQL
- join rules and key normalization rules
- analytical questions and reference answers (SQL + results)
- assumptions, exclusions, and known caveats
- a contribution summary (who did what, which questions were proposed by whom)

The single PDF must be **no more than 2,000 words** (not counting SQL code or data dictionary). I should be able to reproduce your model and query it correctly without guessing table grain or join intent.

---

4. Optional Extension: Dependency Graph *(Bonus)*

> **[OPTIONAL — NOT REQUIRED]** You are not required to implement a dependency graph. If your group decides to pursue it, read this section carefully before committing time to it.

PyPI packages form a directed dependency graph: each release declares the packages it requires, and those targets are themselves packages in the dataset. A correct graph representation in PostgreSQL allows questions about reachability, centrality, and transitive exposure.

If your group chooses to implement the dependency graph extension:

- You may replace **one** of your required 3–4 analytical questions with a graph-based question (e.g. "which packages are reachable transitively from a given vulnerable package?", or "what is the in-degree centrality of the top dependency targets?").
- You may not add extra questions beyond the required 3–4 total. The graph question replaces one, not supplements.
- The implementation must be done entirely in PostgreSQL (e.g. using a recursive CTE or a dedicated adjacency table).
- If your graph model and query are **correct and clearly explained**, your group receives a **3-5 point bonus** on the overall project score.

Partial or incorrect graph implementations do not receive the bonus. If you attempt it and it does not work correctly, you will not be penalised beyond losing the bonus — but you must still have the remained 2–3 questions answered.

---

5. Hints — Suggested Analytical Questions

[HINTS] These are examples of the kind of questions your model should support. You are **not** required to answer these specific questions. Each example below notes which table and field the answer comes from so you can judge difficulty before choosing.

- How many packages published by Google have exactly one declared dependency? *(filter by `ownership.organization`, count `requires_dist` entries per package in your dependency table, keep where count = 1)*
- Which packages have the most security vulnerability records linked to them? *(count vulnerability rows per package in your vulnerability table — a single GROUP BY; no graph traversal needed)*
- Which GitHub organisations own the most packages in the dataset? *(group by `ownership.organization` where non-null, count packages — tests whether you modeled the organisation field cleanly)*
- Which security vulnerabilities cover the widest range of affected package versions? *(count affected-version entries per vulnerability ID in your vulnerability table)*
- Which packages declare the most direct dependencies? *(dependency-related — count rows in your dependency table per source package)*

Appendix A — Data Reference

The starter scripts and SQL referenced below are in the repo: - `fcp/scripts/load_json_to_mongo.py` — loads both JSON files into MongoDB - `fcp/scripts/example_mongo_to_postgres_etl.py` — example ETL from MongoDB → PostgreSQL (`ssc.dim_package`) - `fcp/sql/postgres_dim_package_example.sql` — example `CREATE TABLE` for `ssc.dim_package`

See Section 2 for usage instructions.

A.1 Released Data Files

The two JSON files released to student groups:

Path	Content	Source	Approx. size	Count
`data/pypi_items.json`	Package documents with current metadata, release history, and historical dependency declarations	PyPI JSON API + project enrichment	~4.7 GB	**19,988** package documents
`data/vulnerabilities.json`	Package-level OSV advisory documents, each containing an array of vulnerability records	OSV advisory exports	~28 MB	**780** package documents

A.2 Example ETL Field Mapping

MongoDB field	PostgreSQL table	column	notes
`info.name`	`dim_package`	`package_name`	stored as-is
`info.name` (normalized)	`dim_package`	`normalized_package_name`	lowercase, `-` separator
`info.author_email`	`dim_package`	`author_email`	nullable
`releases.<ver>[].upload_time_iso_8601`	`dim_version`	`uploaded_at`	cast to `timestamptz`
`_historical_requires_dist.<ver>[]`	`fact_dependency`	`dep_name`	one row per declared dependency

A.3 `raw_pypi_package` — MongoDB collection

Each document corresponds to one PyPI package. The `_id` field is the normalized package name. Field names follow the PyPI JSON API schema — refer to that documentation for authoritative field definitions and any fields not listed here.

Field	Type	Description
`_id`	string	Normalized package name (lowercase, `-` separator) used as MongoDB document key

Field	Type	Description
`package_name`	string	Original package name as returned by the PyPI API
`info.name`	string	Canonical package name
`info.version`	string	Latest published version string
`info.summary`	string	One-line package description
`info.author`	string | null	Plain-text author name (separate from email; many packages set one but not the other)
`info.author_email`	string | null	Author contact email (may contain multiple addresses in RFC 5322 format)
`info.requires_dist`	list[string] | null	Dependency specifiers for the latest release (PEP 508 format); null if no dependencies declared
`info.requires_python`	string	Python version constraint for the latest release
`info.license`	string | null	SPDX licence expression or free-text licence name; null for ~32% of packages
`info.classifiers`	list[string]	Trove classifiers (e.g. `Development Status :: 5 - Production/Stable`)
`info.project_urls`	dict	Named URLs (Homepage, Source, Documentation, etc.)
`info.yanked`	bool	Whether the latest release has been yanked
`releases`	dict	Keys are version strings; values are lists of release-file objects
`urls`	list[dict]	Same structure as a `releases` entry; represents the file objects for the latest published version
`releases.<ver>[].upload_time_iso_8601`	string	ISO 8601 upload timestamp with sub-second precision and `Z` suffix — **use this field, not `upload_time`**
`releases.<ver>[].upload_time`	string	Shorter timestamp (no sub-second, no `Z`); also present but less precise — prefer `upload_time_iso_8601`
`releases.<ver>[].packagetype`	string	`sdist`, `bdist_wheel`, etc.
`releases.<ver>[].yanked`	bool	Whether this specific release file has been yanked
`ownership.organization`	string | null	GitHub / PyPI organization that owns the package, if any
`ownership.roles`	list[dict]	List of `{role, user}` objects for package maintainers
`_historical_requires_dist`	dict	Keys are version strings; values are `requires_dist` lists for that historical version — **primary source for dependency edges**

Field	Type	Description
`vulnerabilities`	list[dict]	PyPI-linked vulnerability records for this package (may be empty); each entry contains `id` (e.g. `CVE-...`), `summary`, `details`, `aliases`, `fixed_in`, `link`, `source`, `withdrawn` — **distinct from the OSV collection**

A.4 `raw_osv_package` — MongoDB collection

Each document corresponds to one package that has at least one OSV security advisory (a published vulnerability record). The `_id` field is `<source>:<normalized_package_name>`. Field names within `vulnerabilities[]` follow the OSV schema — refer to that documentation for authoritative field definitions.

Field	Type	Description
`_id`	string	Composite key: `osv-api:<normalized_package_name>`
`package`	string	Package name as it appears in the OSV advisory
`source`	string	Source identifier (e.g. `osv-api`)
`vulnerabilities`	list[dict]	Array of advisory objects (see below)
`vulnerabilities[].id`	string	Advisory ID (e.g. `GHSA-...`, `CVE-...`, `PYSEC-...`)
`vulnerabilities[].summary`	string	One-line description of the vulnerability
`vulnerabilities[].details`	string	Full advisory text in Markdown
`vulnerabilities[].published`	string	ISO 8601 timestamp when the advisory was first published
`vulnerabilities[].modified`	string	ISO 8601 timestamp of the last advisory update
`vulnerabilities[].aliases`	list[string]	Cross-database identifiers (e.g. a GHSA linked to a CVE)
`vulnerabilities[].severity`	list[dict]	CVSS scoring objects: `{type, score}`
`vulnerabilities[].references`	list[dict]	External links: `{type, url}`
`vulnerabilities[].affected`	list[dict]	Affected-package entries with `package`, `versions`, and `ranges` sub-fields
`vulnerabilities[].database_specific`	dict	Source-specific metadata; common keys are `severity` (string label: `CRITICAL`, `HIGH`, `MODERATE`, `LOW`), `cwe_ids` (list), `github_reviewed`, `github_reviewed_at`, `nvd_published_at`; may be an empty dict

References

1. **PyPI JSON API documentation** — https://docs.pypi.org/api/json/
2. **OSV schema documentation** — https://ossf.github.io/osv-schema/
3. **OSV project** — https://osv.dev
4. **PEP 508 — Dependency specification format** — https://peps.python.org/pep-0508/
5. **Graph analytics with recursive CTEs** — https://www.postgresql.org/docs/current/queries-with.html

∞